AIOps: algorithms for operations

Use statistics and machine learning to help run systems: anomaly detection, alert correlation, log and trace patterns, root cause hints, incident assistants and safe auto-remediation.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. What AIOps is and is not (12 min)
2. Anomaly detection on metrics (14 min)
3. Alert correlation and noise reduction (13 min)
4. Logs and traces: finding patterns with ML (14 min)
5. Root cause hints and incident assistants (14 min)
6. Safe automation and auto-remediation (15 min)
7. Debugging and investigating AIOps output (16 min)
8. Telemetry quality: the data AIOps depends on (12 min)
9. Forecasting capacity and saturation (12 min)
10. LLM assistants for incident response (12 min)
11. Measuring AIOps value and avoiding hype (12 min)
12. Runbook automation and guardrails (13 min)
13. Statistics for ops: percentiles, seasonality and correlation (14 min)
14. Hands-on anomaly detection: z-score, MAD and EWMA (16 min)
15. Change-point and trend detection (13 min)
16. Event deduplication, grouping and suppression rules (13 min)
17. Change correlation: linking incidents to deploys (14 min)
18. Topology and dependency-aware root cause (14 min)
19. Tuning alerts with data: precision, recall and burn rates (14 min)
20. Multivariate and service-level anomalies (14 min)
Cheat sheet

Lesson 1. What AIOps is and is not

12 minutes. AIOps applies statistics and machine learning to operations data. Learn what it can do, what it cannot, and how to judge a tool honestly.

AIOps means using algorithms to help run systems: spotting unusual metrics, grouping noisy alerts, pointing at likely causes and automating routine fixes. The promise is less noise and faster recovery. The risk is a black box that adds a new thing to debug. This lesson sets realistic expectations so you can adopt the useful parts and skip the hype.

What you will learn

- Define AIOps and list its common uses
- Separate problems algorithms solve well from those they do not
- Explain why good telemetry comes before any AIOps tool
- Judge a tool by measurable results

What it covers

Four jobs appear again and again: anomaly detection on metrics and logs, alert correlation and noise reduction, root cause hints, and automated or assisted remediation. Newer tools add language models that summarise incidents and search runbooks. Each job is a separate problem with its own limits.

What it needs first

Algorithms can only learn from the data you give them. If metrics are missing, logs are unstructured, service names are inconsistent and changes are not recorded, no tool can recover the picture. Clean telemetry, a service catalogue and a change log are the foundation. Doing that work first often removes much of the need.

Where it helps

It helps where humans cannot keep up: thousands of metrics with daily patterns, alert storms that bury the one that matters, and repetitive checks during an incident. It struggles where the system is new, behaviour changes constantly, or the failure has never happened before.

Where it does not

A model cannot know your business rules, a planned launch or why traffic doubled. It cannot replace on-call judgement. It can mislead when it sounds confident. Treat every output as a hint with a reason you can check, not an answer.

Judging a tool

Define measures before you start: alerts per incident, time to acknowledge, time to find the cause, false positives, missed incidents. Run it beside your current process for a few weeks. Keep it if the numbers improve and engineers trust it. Drop it if they ignore it.

A baseline you can measure

```
$ ./incident_stats.sh --from 2026-01-01 --to 2026-03-31
incidents:                        42
alerts fired:                     6180
alerts per incident (median):     71
pages per week per person:        19
time to acknowledge (median):     6 min
time to identify cause (median): 38 min
false pages (no action needed):   57%
```

Numbers like these are what a tool must improve. Without a baseline, nobody can say whether it helped. The output is a simulated illustration.

MARGA by EknathaLabs marga.eknathalabs.com

How it fails

A tool is bought before telemetry is fixed. Results are not measured, so people argue from opinion. A model flags everything as unusual and is muted. A confident suggestion is trusted without checking.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How What AIOps is and is not fits together

Part	What it does	What goes wrong
What it covers	Four jobs appear again and again: anomaly detection on metrics and logs, alert correlation and noise reduction, root cause hints, and automated or assisted remediation.	Check the Common mistakes list below this diagram for what usually goes wrong here.
What it needs first	Algorithms can only learn from the data you give them.	Buying a tool before fixing telemetry.
Where it helps	It helps where humans cannot keep up: thousands of metrics with daily patterns, alert storms that bury the one that matters, and repetitive checks during an incident.	Where it helps It helps where humans cannot keep up: thousands of metrics with daily patterns, alert storms that bury the one that matters...
Where it does not	A model cannot know your business rules, a planned launch or why traffic doubled.	Trusting confident suggestions without checking.
Judging a tool	Define measures before you start: alerts per incident, time to acknowledge, time to find the cause, false positives, missed incidents.	Check the Common mistakes list below this diagram for what usually goes wrong here.
A baseline you can measure	Numbers like these are what a tool must improve.	Having no baseline to compare against.

Flow: What it covers to What it needs first / What it needs first to Where it helps / Where it helps to Where it does not / Where it does not to Judging a tool / Judging a tool to A baseline you can measure

Incident: A tool that learned the wrong normal

> **Incident:** *A team trained an anomaly detector on three quiet weeks. When a busy sales period began, it flagged every metric as abnormal. The on-call engineers muted it, and later a real fault went unnoticed.*

```
$ aiops-tool status
anomalies open: 412
training window: 2026-02-02 to 2026-02-22
note: no seasonal patterns learned
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. A detector must learn from a window that includes normal busy and quiet periods, and its alert volume must be watched like any other service, or it becomes noise itself.

Interview question

"Would you buy an AIOps platform?" Start with the problem and a baseline, check that telemetry and change data are good, trial it beside current practice, and measure alert noise, time to cause and false positives before deciding.

Common mistakes

- Buying a tool before fixing telemetry
- Having no baseline to compare against
- Trusting confident suggestions without checking
- Training a detector on an unrepresentative window

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Which job is part of AIOps?

Answer: Grouping many alerts into one incident

Why fix telemetry before adopting AIOps?

Answer: Algorithms can only learn from the data they are given

How should you judge an AIOps tool?

Answer: Compare measured results such as alert noise and time to cause with a baseline

What is a risk of AIOps?

Answer: Confident output that nobody checks

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Anomaly detection on metrics

14 minutes. Learn simple baselines, seasonality and thresholds before reaching for complex models, and how to keep false alarms low.

Fixed thresholds fail when normal changes by the hour: a CPU of 70 percent is fine at noon and alarming at 4am. Anomaly detection learns what normal looks like and flags departures. The simplest methods work better than most people expect, and they are easy to explain, which matters when a human must trust the alert at night.

What you will learn

- Compare static thresholds with learned baselines
- Explain seasonality and why it matters
- Apply a rolling mean and standard deviation, and a seasonal baseline
- Reduce false alarms with duration, smoothing and context

Why static thresholds struggle

A single limit ignores daily and weekly rhythm. Set it low and it fires every busy hour. Set it high and it misses a quiet-hour fault. Teams then keep adjusting it, or give up and ignore it. A learned baseline moves with the pattern.

Rolling baselines

Compute the average and spread of a metric over a recent window, then flag values many spreads away. This z-score approach is cheap and easy to explain. It copes poorly with strong daily patterns, because last hour is not a good guide to this hour.

Seasonality

Compare the metric with the same time on previous days or weeks. A Monday 9am value is judged against other Monday 9am values. This single idea removes most false alarms for traffic-like metrics. Add known events, such as launches and holidays, as context.

Cut false alarms

Require the anomaly to last for several minutes, smooth spikes with a median, set a minimum size so tiny absolute changes are ignored, and alert on symptoms users feel, not every internal metric. Review each alert after the fact: fix the rule or delete it.

When to use models

If simple seasonal baselines are not enough, forecasting models or isolation forests can help with many series at once. Start simple, compare the complex approach on past incidents, and keep it only if it finds real problems earlier without adding noise.

A seasonal baseline in PromQL

```
# Requests per second now, compared with the same time last week
(
  sum(rate(http_requests_total{job="checkout"}[5m]))
  /
  sum(rate(http_requests_total{job="checkout"}[5m] offset 1w))
)
# Alert when traffic falls below 60 percent of last week for 10 minutes
# (sum(rate(http_requests_total{job="checkout"}[5m]))
#   / sum(rate(http_requests_total{job="checkout"}[5m] offset 1w))) < 0.6
```

Comparing with the same time last week removes the daily pattern. Pair it with a minimum traffic floor so a quiet hour does not create false alarms.

MARGA by EknathaLabs
marga.eknathalabs.com

How it fails

A model is trained on an unusual period. A launch or holiday looks like an attack. Alerts fire on tiny changes in low-traffic services. Nobody reviews the detector, so false alarms pile up until it is ignored.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Anomaly detection on metrics fits together

Part	What it does	What goes wrong
Why static thresholds struggle	A single limit ignores daily and weekly rhythm.	Comparing with the previous hour only.
Rolling baselines	Compute the average and spread of a metric over a recent window, then flag values many spreads away.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Seasonality	Compare the metric with the same time on previous days or weeks.	Ignoring holidays and launches.
Cut false alarms	Require the anomaly to last for several minutes, smooth spikes with a median, set a minimum size so tiny absolute changes are ignored, and alert on symptoms users feel...	Check the Common mistakes list below this diagram for what usually goes wrong here.
When to use models	If simple seasonal baselines are not enough, forecasting models or isolation forests can help with many series at once.	Never reviewing past alerts.
A seasonal baseline in PromQL	Comparing with the same time last week removes the daily pattern.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Why static thresholds struggle to Rolling baselines / Rolling baselines to Seasonality / Seasonality to Cut false alarms / Cut false alarms to When to use models / When to use models to A seasonal baseline in PromQL

Incident: The Monday that looked like an outage

> **Incident:** *A detector compared each hour with the previous hour. Every Monday morning, when traffic rose sharply, it raised a high-severity alert. On-call engineers learned to ignore Monday alerts, and one Monday a real fault was missed.*

```
$ detector explain --alert 4412
metric: checkout_requests  value: 1840/s
compared with: previous hour mean 620/s (z = 6.2)
seasonal expectation (same hour, last 4 Mondays): 1790/s (z = 0.3)
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Comparing with the same hour on previous Mondays would have shown normal behaviour. Showing both comparisons also made the alert explainable.

Interview question

"How would you alert on traffic anomalies?" Compare with a seasonal baseline from the same time on earlier weeks, require the shift to last, ignore tiny absolute values, and review false alarms until the rule is quiet.

Common mistakes

- Comparing with the previous hour only
- Alerting on a single data point
- Ignoring holidays and launches
- Never reviewing past alerts

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Why do static thresholds often fail?

Answer: Normal values change with time of day and week

What does a seasonal baseline compare with?

Answer: The same time on earlier days or weeks

Which reduces false alarms?

Answer: Requiring the anomaly to last for several minutes

What should you try before complex models?

Answer: Simple rolling and seasonal baselines

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Alert correlation and noise reduction

13 minutes. Group related alerts into one incident, remove duplicates and use dependencies to point at the source of a storm.

During a real outage, one failing database can trigger hundreds of alerts from every service that depends on it. The person on call sees a wall of red and has to work out which alert is the cause and which are echoes. Correlation groups related alerts, and noise reduction removes the ones that never needed a human. Both can be done well with plain rules before adding machine learning.

What you will learn

- Explain alert storms and why they slow response
- Deduplicate and group alerts using labels and time
- Use service dependencies to find the likely source
- Measure noise and track it down over time

Why storms hurt

Each extra alert costs attention. When many fire at once, the engineer must read, sort and guess. The first alert is often the most useful, and it gets buried. A good system shows one incident with the likely cause on top and the echoes folded underneath.

Deduplicate and group

Give every alert stable labels: service, environment, team and a fingerprint of the problem. Identical alerts collapse into one with a count. Alerts that share a service or cluster and start within a few minutes group together. This alone often cuts volume by more than half.

Use topology

If you know that checkout calls payments, and payments calls the database, then a database fault explains alerts above it. Walk the dependency graph from the alerting services toward the shared dependency that is also unhealthy. Surface that one first and label the rest as downstream.

Inhibit and silence by design

Inhibition rules suppress low-value alerts while a high-value one is active, for example, mute pod alerts when the whole node is down. Planned work should silence alerts with an expiry and a reason. Avoid permanent silences: they hide new problems.

Measure and prune

Track alerts per incident, the share that led to an action, and alerts that always resolve alone. Retire or retune any alert that is mostly ignored. Noise is a debt that grows quietly, so give someone ownership of the number.

Grouping and inhibition in Alertmanager

```
route:
  receiver: team-pager
  group_by: [cluster, service]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
inhibit_rules:
  - source_matchers: [alertname = "NodeDown"]
    target_matchers: [severity = "warning"]
    equal: [cluster, node]
  - source_matchers: [alertname = "DatabaseDown"]
    target_matchers: [alertname =~ "Service.*Errors"]
    equal: [cluster]
receivers:
  - name: team-pager
```

MARGA by EknathaLabs
marga.eknathalabs.com

Alerts for the same cluster and service are grouped for 30 seconds before the first page. When a database is down, service error alerts in the same cluster are inhibited so the cause stands out.

How it fails

Alerts have no labels, so nothing can be grouped. Silences never expire. Inhibition hides a real second fault. Dependency data is out of date, so the cause shown is wrong. Nobody measures noise.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Alert correlation and noise reduction fits together

Part	What it does	What goes wrong
Why storms hurt	Each extra alert costs attention. When many fire at once, the engineer must read, sort and guess.	No ownership of alert noise.
Deduplicate and group	Give every alert stable labels: service, environment, team and a fingerprint of the problem.	Alerts without consistent labels.
Use topology	If you know that checkout calls payments, and payments calls the database, then a database fault explains alerts above it.	Inhibition rules that hide real faults.
Inhibit and silence by design	Inhibition rules suppress low-value alerts while a high-value one is active, for example, mute pod alerts when the whole node is down.	Permanent silences.
Measure and prune	Track alerts per incident, the share that led to an action, and alerts that always resolve alone.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Grouping and inhibition in	Alerts for the same cluster and service are grouped for 30 seconds before the first page.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Why storms hurt to Deduplicate and group / Deduplicate and group to Use topology / Use topology to Inhibit and silence by design / Inhibit and silence by design to Measure and prune / Measure and prune to Grouping and inhibition in

Incident: Two hundred pages for one disk

> **Incident:** *A shared storage volume filled up and many services began failing. The on-call engineer received around two hundred alerts in ten minutes and spent the first part of the incident reading them instead of looking at the disk.*

```
$ alertctl summary --window 10m
alerts received: 214
distinct fingerprints: 31
grouped into incidents: 2
likely source: storage-vol-7 (disk 100%), dependents: 29
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Grouping by service and dependency turns two hundred alerts into two incidents with one named likely source.

Interview question

"How would you reduce alert noise during an outage?" Add stable labels, deduplicate, group by service and cluster, use dependency data to show the likely root, add inhibition rules, and track alerts per incident as a metric.

Common mistakes

- Alerts without consistent labels
- Permanent silences

- Inhibition rules that hide real faults
- No ownership of alert noise

Check yourself

What is an alert storm?

Answer: Many alerts firing at once from one underlying fault

What makes grouping possible?

Answer: Stable labels such as service, cluster and team

How does dependency data help?

Answer: It points to the shared dependency that explains alerts above it

Why avoid permanent silences?

Answer: They can hide new, real problems

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Logs and traces: finding patterns with ML

14 minutes. Cluster log lines into templates, spot new or rare messages and link behaviour changes to deployments.

Logs are huge and mostly boring. The interesting line is often one you have never seen before. Machine learning and simple statistics can help you find it: group similar lines into templates, count them, and highlight what is new or suddenly frequent. Traces add timing, so slow outliers and changed paths become visible. As always, structured data makes the work easier.

What you will learn

- Explain log templates and why clustering them helps
- Detect new and rare messages after a change
- Use trace data to spot slow or unusual paths
- Link patterns to deployments and configuration changes

From lines to templates

Many log lines differ only in values: "user 4412 timed out after 3000 ms" and "user 8123 timed out after 2900 ms". Replace the variable parts with placeholders and you get one template. Counting templates turns millions of lines into a short list you can scan and compare over time.

New and rare messages

After a release, list templates that did not exist before, and templates whose rate jumped. New error templates right after a deployment are strong hints. Rare messages at the start of an incident are worth reading first, because they tend to be the first sign of change.

Count by version and host

Group counts by software version, region and host. If a template appears only on version 2.4.1, you have narrowed the search before reading a single line. This works with plain queries, with no model at all, if your logs carry the version label.

Traces find the slow path

Traces record each step of a request with its duration. Compare the slowest requests with typical ones: which service, call or database query takes the extra time? Look for a changed path, such as a new retry or a call that used to be cached. The span tells you where.

Tie it to change

Overlay deployments, configuration pushes and feature flag changes on your charts. When a pattern starts at the minute of a change, you have a candidate cause. The change log is the most valuable context for any detection method, and it costs very little to keep.

New templates since the last release

```
$ logstat templates --service checkout --since release:2.4.1 --new
count  template
912    payment gateway timeout after <NUM> ms, retry <NUM> of <NUM>
87     cache miss for key <KEY>, falling back to database
3      unexpected status <NUM> from tax service
$ logstat compare --version 2.4.0 2.4.1 --top 3
payment gateway timeout: 0/min -> 41/min
```

The tool lists message templates that first appeared after release 2.4.1. A jump from zero to 41 timeouts a minute narrows the search to one dependency. The output is a simulated illustration.

MARGA by EknathaLabs marga.eknathalabs.com

How it fails

Logs have no version or request id, so grouping is impossible. A clustering model merges unrelated messages. Sensitive data in logs gets copied into new systems. Sampling drops the rare line you needed.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Logs and traces: finding patterns with ML fits together

Part	What it does	What goes wrong
From lines to templates	Many log lines differ only in values: "user 4412 timed out after 3000 ms" and "user 8123 timed out after 2900 ms".	Logs with no version, host or request id.
New and rare messages	After a release, list templates that did not exist before, and templates whose rate jumped.	Sampling away rare errors.
Count by version and host	Group counts by software version, region and host.	Trusting clusters without reading examples.
Traces find the slow path	Traces record each step of a request with its duration.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Tie it to change	Overlay deployments, configuration pushes and feature flag changes on your charts.	Check the Common mistakes list below this diagram for what usually goes wrong here.
New templates since the last release	The tool lists message templates that first appeared after release 2.4.1.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: From lines to templates to New and rare messages / New and rare messages to Count by version and host / Count by version and host to Traces find the slow path / Traces find the slow path to Tie it to change / Tie it to change to New templates since the last release

Incident: The error nobody had seen before

> **Incident:** *After a release, a new error line appeared at a low rate. It was buried among millions of lines and nobody noticed until the rate grew and customers complained. The line had never appeared before the release.*

```
$ logstat templates --service orders --since release:5.1.0 --new
count  template
64     serialization failed for field <NAME> in schema v<NUM>
$ logstat timeline --template 4f2a --bucket 10m
10:00 2   10:10 7   10:20 19   10:30 64
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. A simple report of templates that are new since a release would have surfaced the line within minutes of the deployment, long before it became a customer problem.

Interview question

"How do you find the needle in millions of log lines?" Group lines into templates, list new and rapidly growing ones after a change, group by version and host, and confirm with traces and the change log.

Common mistakes

- Logs with no version, host or request id
- Sampling away rare errors
- Sending sensitive data into analysis tools
- Trusting clusters without reading examples

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What is a log template?

Answer: A line with variable values replaced by placeholders

Which is a strong hint after a deployment?

Answer: A message template that did not exist before the release

How do traces help?

Answer: They show which step of a request takes the extra time

Why overlay changes on charts?

Answer: A pattern starting at the minute of a change points to a candidate cause

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Root cause hints and incident assistants

14 minutes. Rank likely causes from changes and dependencies, and use language models to summarise safely while a human stays in charge.

During an incident the hardest question is what changed. Tools can speed the search by lining up recent changes, dependency health and anomalies, then ranking suggestions. Language models can add readable summaries and runbook search. The risk is false confidence: a plausible explanation that is wrong can send the team down the wrong path. This lesson shows how to use assistants as a fast second pair of eyes with evidence attached.

What you will learn

- Rank candidate causes from change events, timing and dependencies
- Use language models for summaries and search with guardrails
- Require evidence and a confidence reason for each hint
- Keep a human decision at every step that changes production

Start from change

Most incidents follow a change: a deployment, a configuration push, a flag, a certificate expiry or a traffic shift. List every change in the window before the first symptom, ordered by closeness in time and by relation to the failing service. Even without any model, this list is very effective.

Add dependency health

Overlay the dependency graph. A service whose dependency started failing a moment earlier is a better suspect than one with no relation. Combine closeness in time, closeness in the graph and the size of the anomaly into a simple score, and show the reasons next to the rank.

Language models as summarisers

A model can read the timeline, alerts and chat and write a clear status update, or find the matching runbook. It is good at language, and weaker at being right. Give it the evidence, ask it to quote sources, and show the quotes. Never let it invent a cause from nothing.

Guardrails

Keep incident data private: send only what the policy allows. Label every model output as a suggestion. Log what was asked and answered for later review. Do not give the model write access to production. If it suggests an action, a person approves and runs it.

Learn from outcomes

After each incident, record the real cause and whether the hint was right, missed or misleading. Measure how often the top three hints contained the cause. That number tells you whether to trust the tool more, tune it or turn it off.

A hint report with evidence

```
$ incident hints --service checkout --since 14:02
rank score  hint
1    0.82   deploy payments 3.9.0 at 13:58 (4 min before; payments is a dependency)
2    0.55   config change: payment_timeout_ms 3000 -> 800 at 13:57
3    0.21   certificate renewal for api.example.com at 12:10
evidence for 1: errors on payments rose 13:59, checkout errors followed 14:02
note: hints are suggestions, confirm before acting
```

Each hint shows its time gap, relation and evidence, so an engineer can confirm or reject it quickly. The output is a simulated illustration.

MARGA by EknathaLabs
marga.eknathalabs.com

How it fails

A model states a cause with no evidence. Private data is pasted into an outside service. The assistant is allowed to run commands. The team stops thinking and follows hints. Nobody records whether the hints were right.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Root cause hints and incident assistants fits together

Part	What it does	What goes wrong
Start from change	Most incidents follow a change: a deployment, a configuration push, a flag, a certificate expiry or a traffic shift.	Trusting a hint without checking evidence.
Add dependency health	Overlay the dependency graph. A service whose dependency started failing a moment earlier is a better suspect than one with no relation.	Pasting private data into tools without a policy.
Language models as summarisers	A model can read the timeline, alerts and chat and write a clear status update, or find the matching runbook.	Giving an assistant write access to production.
Guardrails	Keep incident data private: send only what the policy allows.	Do not give the model write access to production.
Learn from outcomes	After each incident, record the real cause and whether the hint was right, missed or misleading.	Never scoring hints against real causes.
A hint report with evidence	Each hint shows its time gap, relation and evidence, so an engineer can confirm or reject it quickly.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Start from change to Add dependency health / Add dependency health to Language models as summarisers / Language models as summarisers to Guardrails / Guardrails to Learn from outcomes / Learn from outcomes to A hint report with evidence

Incident: The confident wrong answer

> **Incident:** *During an outage an assistant suggested that a database was the cause. The team spent twenty minutes on the database. The real cause was a configuration change on a load balancer, which had been in the change list but ranked low.*

```
$ incident hints --service shop --since 09:40
1  0.74  database latency up at 09:41
2  0.39  load balancer config push at 09:39
$ incident timeline --service shop
09:39 lb config push   09:41 db latency up (caused by retry storm)
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. The ranking looked at symptoms, not at order of events. Showing the timeline next to the hints let the team see that the database was a result, not the cause.

Interview question

"Would you let an AI assistant run incident response?" It can summarise, search runbooks and rank hints with evidence, but people decide and approve any change to production, and every output is a suggestion to verify.

Common mistakes

- Trusting a hint without checking evidence
- Pasting private data into tools without a policy

MARGA by EknathaLabs marga.eknathalabs.com

- Giving an assistant write access to production
- Never scoring hints against real causes

Check yourself

What is a good first step in finding a cause?

Answer: List the changes made just before the first symptom

What should an assistant show with each hint?

Answer: The evidence and reasons behind it

Who approves a production change an assistant suggests?

Answer: A person

Why score hints after incidents?

Answer: To learn how often they contain the real cause and decide how far to trust them

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Safe automation and auto-remediation

15 minutes. Automate routine fixes with limits, approvals and rollbacks, so automation reduces toil without becoming the cause of the next outage.

Many incidents end with the same few commands: restart a service, clear a disk, scale up, fail over. Automating them saves time and sleep. But automation acts fast and at scale, so a wrong rule can make an incident bigger in seconds. This lesson covers how to start small, add limits and keep humans in control until the evidence says otherwise.

What you will learn

- Choose which fixes are safe to automate
- Add limits, cooldowns and approvals
- Design for rollback and visibility
- Measure whether automation helps

Pick the right candidates

Good candidates are frequent, well understood, low risk and easy to verify: restarting a known stuck worker, rotating a log, clearing a cache. Poor candidates are rare, high impact or poorly understood, such as database failover or deleting data. Start from your runbooks and incident history.

Verify before you act

Before the action, check the condition again: is the alert still firing, is the target the right one, is a change in progress? After the action, check that the problem is solved. If it is not, stop and escalate to a person instead of trying again and again.

Limits that contain mistakes

Set a maximum number of actions per hour, a cooldown between actions, a cap on how many targets at once and a blast radius rule such as never more than ten percent of a fleet. A global kill switch stops every automation in one step.

Approval levels

Introduce automation in stages: suggest the action, then run it with one-click approval, then run it unattended for the safest cases. Move up a level only after a run of correct decisions, and always keep the audit log.

Visibility and rollback

Every automated action writes who or what did it, why, what it changed and how to undo it, in the incident channel and the audit log. Prefer actions that can be reversed. For those that cannot, require approval.

MARGA by EknathaLabs
marga.eknathalabs.com

A remediation rule with limits

```
name: restart-stuck-worker
trigger:
  alert: WorkerQueueStalled
  for: 10m
preconditions:
  - alert_still_firing: true
  - no_active_deploy: true
  - target_count_max: 2
action:
  run: kubectl rollout restart deployment/worker -n jobs
limits:
  max_per_hour: 2
  cooldown: 30m
  kill_switch: automation/global
verify:
  after: 5m
  expect: queue_depth < 100
on_verify_fail: page_human
```

The rule only acts if the alert is still active and no deployment is running. It is limited to two restarts an hour and escalates to a person if the queue does not recover.

How it fails

A rule has no cooldown and loops. It acts on the wrong environment. It hides a recurring problem because it always fixes the symptom. A kill switch does not exist. Nobody reviews what the automation did.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Safe automation and auto-remediation fits together

Part	What it does	What goes wrong
Pick the right candidates	Good candidates are frequent, well understood, low risk and easy to verify: restarting a known stuck worker, rotating a log, clearing a cache.	Automating rare, risky actions first.
Verify before you act	Before the action, check the condition again: is the alert still firing, is the target the right one, is a change in progress?	No verification after the action.
Limits that contain mistakes	Set a maximum number of actions per hour, a cooldown between actions, a cap on how many targets at once and a blast radius rule such as never more than ten percent of a...	No cooldown or action limits.
Approval levels	Introduce automation in stages: suggest the action, then run it with one-click approval, then run it unattended for the safest cases.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Visibility and rollback	Every automated action writes who or what did it, why, what it changed and how to undo it, in the incident channel and the audit log.	For those that cannot, require approval.
A remediation rule with limits	The rule only acts if the alert is still active and no deployment is running.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Pick the right candidates to Verify before you act / Verify before you act to Limits that contain mistakes / Limits that contain mistakes to Approval levels / Approval levels to Visibility and rollback / Visibility and rollback to A remediation rule with limits

Incident: The cleanup that deleted too much

Incident: *An automation was meant to delete old temporary files when a disk filled. A path variable was empty, so the command ran against a much larger directory, and it deleted data that other services needed.*

MARGA by EknathaLabs

```
$ remediate run cleanup-tmp --dry-run
target path: ""   (EMPTY)
resolved delete scope: /var/data/*
result: ABORT, target path empty
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. A dry run, a check that the target is not empty and a rule that deletes only inside a named directory would have stopped the action before it did any harm.

Interview question

"What would you automate first on-call?" Frequent, low-risk, easily verified fixes, with preconditions, limits, a verify step, an audit log, a kill switch and a path from suggestion to approval to unattended mode.

Common mistakes

- Automating rare, risky actions first
- No cooldown or action limits
- No verification after the action
- Hiding recurring faults instead of fixing them

Check yourself

Which fix is a good first candidate for automation?

Answer: Restarting a known stuck worker

Why add a cooldown?

Answer: To stop the same action repeating in a loop

What should happen if verification fails?

Answer: Stop and escalate to a person

Why keep an audit log of automated actions?

Answer: People can see what was done, why and how to undo it

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Debugging and investigating AIOps output

16 minutes. How to check an AI assistant's answers during an incident: confirm the evidence, test the claim against data, and keep a person in charge of the action.

An AIOps tool can summarise alerts, suggest causes and propose fixes. Its answer is a hypothesis. Treat it the way you would treat a new colleague's guess: useful, worth checking, and not yet a fact.

What you will learn

- Check an AI summary against the source evidence
- Spot a confident answer that has no support
- Compare a suggested cause with a timeline of changes
- Record misses so the tool and its limits improve

Ask for the evidence

Every claim in a summary should point to something: a log line, a metric, a change record. If it does not, ask for the source. A claim with no source is a guess that happens to be well written.

Test the claim

Turn the suggested cause into a question you can check in the data. If it says a deploy caused the errors, compare the error rate before and after the deploy time. If the data disagrees, the suggestion is wrong, however fluent.

Compare with the timeline

Line up alerts, deploys, config changes and traffic on one timeline. A cause must come before its effects. Suggestions that name something that happened later, or in a different service, are suspect.

Watch the false positives and misses

Count how often a suggestion was right, wrong, or missing. Keep the list. It tells you which alerts to trust, and it is the evidence for tuning or turning a feature off.

Keep a person on the action

Reading is cheap and reversible. Restarting, scaling or rolling back is not. A person approves each action, with the evidence in front of them, and the tool logs who approved it.

Checking a claim against the data

```
$ promtool query instant http://prometheus:9090 'sum(rate(http_requests_total{code=~"5.."}[5m]))'
5xx rate now: 0.2 per second
$ git log --since="2 hours ago" --oneline
a41f2c2 reduce cache size
$ kubectl rollout history deployment/checkout
REVISION  CHANGE-CAUSE
41        reduce cache size
```

The assistant blamed a deploy, but the error rate is low and the only recent change is a cache setting. The first look does not support rolling back. The output is a simulated illustration.

How it fails

The summary is accepted because it reads well. The suggested action is run with no check. Wrong suggestions are not recorded, so the same error repeats. The tool is given write access too early.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Debugging and investigating AIOps output fits together

Part	What it does	What goes wrong
Ask for the evidence	Every claim in a summary should point to something: a log line, a metric, a change record.	Accepting a fluent summary without evidence.
Test the claim	Turn the suggested cause into a question you can check in the data.	Running a suggested action with no check.
Compare with the timeline	Line up alerts, deploys, config changes and traffic on one timeline.	Not recording wrong suggestions.
Watch the false positives and misses	Count how often a suggestion was right, wrong, or missing.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Keep a person on the action	Reading is cheap and reversible. Restarting, scaling or rolling back is not.	Giving the tool write access before it is trusted.
Checking a claim against the data	The assistant blamed a deploy, but the error rate is low and the only recent change is a cache setting.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Ask for the evidence to Test the claim / Test the claim to Compare with the timeline / Compare with the timeline to Watch the false positives and misses / Watch the false positives and misses to Keep a person on the action / Keep a person on the action to Checking a claim against the data

Incident: The confident rollback

> **Incident:** *During a slow-checkout alert an assistant named a deploy as the cause. The on-call engineer rolled back without checking. The real cause was a database lock, and the rollback added ten minutes of disruption.*

```
$ promtool query instant http://prometheus:9090 'max(pg_locks_waiting)'
pg_locks_waiting 41
(this is a simulated illustration, not a real incident)
```

The output is a simulated illustration. Testing the claim against the metrics and the timeline would have shown the locks and avoided a rollback that was never needed.

Interview question

"Would you let an AIOps tool fix incidents by itself?" Not at first. I would let it read and suggest, check each claim against evidence, keep a person approving every action, record hits and misses, and widen its scope only where it has earned trust.

Common mistakes

- Accepting a fluent summary without evidence
- Running a suggested action with no check
- Not recording wrong suggestions
- Giving the tool write access before it is trusted

Check yourself

How should you treat an AI suggestion about a cause?

Answer: As a hypothesis to test against the data

What makes a claim in a summary trustworthy?

Answer: It points to evidence you can open and check

MARGA by EknathaLabs marga.eknathalabs.com

Why compare a suggested cause with a timeline?

Answer: A cause must come before its effects

Why record the tool's wrong suggestions?

Answer: They show which suggestions to trust and justify tuning or limits

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Telemetry quality: the data AIOps depends on

12 minutes. Fix the data before the model: consistent names, labels, timestamps and completeness so machine learning on operations data can work.

Machine learning finds patterns in what you give it. If service names differ between tools, timestamps drift or half the metrics are missing, the result is confident nonsense. The first AIOps project is usually a data project.

What you will learn

- Naming and labeling conventions that join data together
- Time, clocks and missing data
- A service catalog as the common key
- Measuring data quality

Consistent names

Use one name per service across metrics, logs, traces and tickets, with standard labels such as `service`, `env`, `team` and `version`. Follow common conventions rather than inventing your own.

```
# OpenTelemetry resource attributes used everywhere
service.name: checkout
service.version: 2.8.1
deployment.environment: prod
team: payments
```

Time and gaps

Correlating events needs clocks that agree. Use NTP or chrony, store timestamps in UTC and check skew. Gaps matter too: a metric that stops reporting looks like zero or like an anomaly, depending on the tool.

```
# which scrape targets stopped reporting
up == 0
# metrics with too many label combinations
topk(10, count by (__name__) ({__name__=~".+"}))
```

Catalog and ownership

A **service catalog** lists every service with owner, tier, dependencies and runbook. Telemetry and incident tools use it to join signals and to route alerts. Without it, machine learning cannot tell which team to notify.

Measure quality

Track the share of services with complete labels, the number of unnamed or duplicate services, scrape success rate and log parsing success. Treat falling data quality like any other incident source, and review it monthly.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Telemetry quality: the data AIOps depends on fits together

Part	What it does	What goes wrong
Consistent names	Use one name per service across metrics, logs, traces and tickets, with standard labels such as service, env, team and version.	Different names for one service.
Time and gaps	Correlating events needs clocks that agree.	Clock skew between hosts.
Catalog and ownership	A service catalog lists every service with owner, tier, dependencies and runbook.	Buying AIOps tools before fixing data.
Measure quality	Track the share of services with complete labels, the number of unnamed or duplicate services, scrape success rate and log parsing success.	No owner metadata on services.

Flow: Consistent names to Time and gaps / Time and gaps to Catalog and ownership / Catalog and ownership to Measure quality

Incident: Five names for one service

Incident: *The same service appeared under different names in metrics, logs and traces.*

```
metrics:   service="checkout-svc"
logs:      app="Checkout"
traces:    service.name="checkout_api"
incidents: component="payments-checkout"
```

Correlation produced nothing because the data would not join. The team created a catalog with one canonical name, mapped old names for a transition period and enforced labels in the collection pipeline.

Common mistakes

- Buying AIOps tools before fixing data
- Different names for one service
- Clock skew between hosts
- No owner metadata on services

Check yourself

What does a service catalog provide?

Answer: A common key and ownership

Why check clocks?

Answer: Correlating events needs agreeing time

What does a missing metric look like to a model?

Answer: Zero or an anomaly

What is a good first AIOps project?

Answer: Fixing data quality

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Forecasting capacity and saturation

12 minutes. Forecast disk, memory and traffic growth with simple trend and seasonality models and alert before you run out.

Capacity incidents are predictable because resources grow in patterns. A little forecasting turns surprises into planned work, and you do not need machine learning for the first big wins.

What you will learn

- Time to full as a better alert than percent used
- Trend and seasonality in operational metrics
- Forecasting traffic and events you know about
- Measuring forecast error and avoiding false confidence

Time to full

```
# will this disk fill within 7 days, based on the last 6 hours of growth?
predict_linear(node_filesystem_avail_bytes{mountpoint="/data"}[6h], 7 * 24 * 3600) < 0
```

Alerting on `> 85 percent` fires for slow, harmless usage and misses fast growth. Alerting on **time to full** adapts to the actual rate.

Trend and seasonality

Most traffic has daily and weekly cycles. A simple approach is to compare with the same hour last week and to fit a trend on the weekly average.

```
import numpy as np
weeks = np.array([410, 428, 445, 470, 492, 515])      # weekly peak requests/s
slope, intercept = np.polyfit(range(len(weeks)), weeks, 1)
print("next 8 weeks:", [round(slope * (len(weeks) + i) + intercept) for i in range(1, 9)])
```

Known events

Add launches, campaigns and holidays as separate adjustments rather than letting the model treat them as noise. Ask product teams for upcoming events each quarter.

Check the forecast

Record each forecast and compare with what happened. If error is large, widen the safety margin or use a different method. Show a range, not one number, and review the main forecasts with the owners. Plan lead times: ordering hardware or raising a cloud quota can take longer than the forecast horizon.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Forecasting capacity and saturation fits together

Part	What it does	What goes wrong
Time to full	Alerting on > 85 percent fires for slow, harmless usage and misses fast growth.	Alerting only on percent used.
Trend and seasonality	Most traffic has daily and weekly cycles.	Ignoring seasonality and known events.
Known events	Add launches, campaigns and holidays as separate adjustments rather than letting the model treat them as noise.	Forecast horizons shorter than procurement lead time.
Check the forecast	Record each forecast and compare with what happened.	Forecasts with no error tracking.

Flow: Time to full to Trend and seasonality / Trend and seasonality to Known events / Known events to Check the forecast

MARGA by EknathaLabs marga.eknathalabs.com

Incident: Seventy percent and nine days left

Incident: *A disk looked healthy but was growing quickly.*

```
/data  70% used  (1.4 TB free)
growth: 38 GB/day over the last 6h
predicted full in 9.1 days
```

A fixed percentage threshold said everything was fine. The forecast alert warned a week ahead, and the team added retention for a runaway log table before the weekend.

Common mistakes

- Alerting only on percent used
- Ignoring seasonality and known events
- Forecasts with no error tracking
- Forecast horizons shorter than procurement lead time

Check yourself

Why alert on time to full?

Answer: It adapts to the growth rate

What does predict_linear do?

Answer: Extrapolates a trend

How treat a planned launch?

Answer: Add it as a known adjustment

What check keeps forecasts honest?

Answer: Compare with actuals

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. LLM assistants for incident response

12 minutes. Use language models to summarize incidents, draft updates and search runbooks, with guardrails for accuracy, privacy and human approval.

During an incident, people read too much and write too little. Language models are good at turning a wall of text into a summary, and at drafting updates, which frees humans to think. They are bad at being right without evidence.

What you will learn

- Safe uses: summaries, timelines, drafts and search
- Grounding answers in your runbooks and data
- Privacy, redaction and access control
- Human approval and review

Good tasks

- Summarize a long chat or log excerpt into a few lines.
- Draft a **timeline** from alerts, deploys and messages.
- Draft **stakeholder updates** for an engineer to edit and send.
- Search runbooks and past incidents for similar cases.
- Write the first draft of a **postmortem** from the timeline.

Ground the answers

```
def answer(question, search, llm):
    docs = search(question, top_k=5)        # runbooks, past incidents, service docs
    context = "\n\n".join(f"[{d.id}] {d.text}" for d in docs)
    prompt = (
        "Answer only from the context. Cite ids like [runbook-12]. "
        "If the answer is not there, say you do not know.\n\n"
        f"Context:\n{context}\n\nQuestion: {question}"
    )
    return llm(prompt)
```

Require citations and refuse answers without sources. Show the source text next to the answer so a person can verify in seconds.

Privacy

Redact secrets, tokens and personal data before sending text to a model, use an approved provider with the right data terms and limit which channels the assistant can read. Keep an audit log of prompts and responses.

Keep humans in charge

The assistant **suggests**, the engineer **decides**. Never let it run commands, change configuration or page people without explicit approval. Measure usefulness with feedback buttons and review wrong answers in the postmortem, since they teach you where grounding is weak.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How LLM assistants for incident response fits together

Part	What it does	What goes wrong
Good tasks	Summarize a long chat or log excerpt into a few lines.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Ground the answers	Require citations and refuse answers without sources.	Answers without cited sources.
Privacy	Redact secrets, tokens and personal data before sending text to a model, use an approved provider with the right data terms and limit which channels the assistant can...	Sending secrets or customer data to a model.
Keep humans in charge	The assistant suggests, the engineer decides.	Not reviewing wrong answers.

Flow: Good tasks to Ground the answers / Ground the answers to Privacy / Privacy to Keep humans in charge

Incident: The command that did not exist

Incident: *An assistant suggested a made-up flag during an outage.*

```
$ dbctl failover --force-safe
error: unknown flag: --force-safe
elapsed: 6 minutes lost to the wrong suggestion
```

The assistant answered from general knowledge instead of the team's tools. The engineer noticed, but time was lost. The team connected it to the real runbooks, required citations and made it say it did not know when nothing matched.

Common mistakes

- Letting a model run commands unsupervised
- Answers without cited sources
- Sending secrets or customer data to a model
- Not reviewing wrong answers

Check yourself

What should an incident assistant cite?

Answer: Runbook or incident sources

Who decides actions?

Answer: The engineer

What should happen before sending text to a model?

Answer: Redact secrets and personal data

What good task suits an assistant?

Answer: Drafting a timeline

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Measuring AIOps value and avoiding hype

12 minutes. Decide whether an AIOps tool helps with before and after metrics such as noise, time to detect and time to resolve, and run honest pilots.

Every AIOps product promises less noise and faster recovery. Your job is to test those claims on your own data, with measures that capture both the benefits and the dangers.

What you will learn

- Choosing metrics: precision, recall, MTTD and MTTR
- Shadow mode and control groups
- Comparing against a simple baseline
- Counting total cost and risk

Metrics

- **Precision:** of the alerts it raised, how many were real.
- **Recall:** of the real problems, how many it caught.
- **MTTD and MTTR:** mean time to detect and to resolve.
- **Alert volume and pages per on-call shift.**
- **Human effort:** time spent tuning, labeling and handling its mistakes.

```
def precision_recall(tp, fp, fn):
    return tp / (tp + fp), tp / (tp + fn)

print(precision_recall(tp=42, fp=8, fn=6))   # (0.84, 0.875)
```

Run an honest pilot

Start in **shadow mode**: the tool runs and records what it would do, but humans still handle incidents. Compare its decisions with what really happened over several weeks, including at least one busy period. If possible use a control group, such as one team with the tool and one without.

Baselines

Compare with a simple rule, such as a static threshold with a sensible window, or a plain deduplication rule. If a simple baseline gets most of the benefit, choose it.

Total cost and risk

Include licences, compute, integration work, tuning time, and the cost of a missed incident. Define a **rollback** plan and review results quarterly. Stop or shrink anything that does not show a measurable gain.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Measuring AIOps value and avoiding hype fits together

Part	What it does	What goes wrong
Metrics	Precision: of the alerts it raised, how many were real.	Judging only by alert reduction.
Run an honest pilot	Start in shadow mode: the tool runs and records what it would do, but humans still handle incidents.	Skipping a shadow period.
Baselines	Compare with a simple rule, such as a static threshold with a sensible window, or a plain deduplication rule.	No comparison with a simple rule.
Total cost and risk	Include licences, compute, integration work, tuning time, and the cost of a missed incident.	Ignoring the work of maintaining the tool.

Flow: Metrics to Run an honest pilot / Run an honest pilot to Baselines / Baselines to Total cost and risk

Incident: Ninety percent less noise, two missed incidents

Incident: *Shadow mode revealed that suppression hid real incidents.*

```
alerts before: 1820/week
alerts after:  182/week (-90%)
real incidents suppressed during shadow period: 2
```

The headline number counted all noise reduction as good. Looking at missed incidents showed the tool suppressed two real problems because they looked like usual noise. The team used the tool only to group related alerts and kept humans in charge of suppression.

Common mistakes

- Judging only by alert reduction
- Skipping a shadow period
- No comparison with a simple rule
- Ignoring the work of maintaining the tool

Check yourself

What does recall measure?

Answer: Real problems it caught

What is shadow mode?

Answer: The tool records decisions without acting

Why compare with a baseline?

Answer: A simple rule may do as well

Which cost is often forgotten?

Answer: Human tuning and maintenance

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Runbook automation and guardrails

13 minutes. Turn runbooks into safe automation with preconditions, dry runs, rate limits, approvals and rollback, and grow trust step by step.

Runbooks are the natural first automation target because the steps are already known. The danger is acting on bad information, so safe automation is mostly about checks, limits and the ability to undo.

What you will learn

- The ladder from manual steps to auto-remediation
- Preconditions and independent signals
- Dry runs, rate limits and circuit breakers
- Logging, rollback and review

The ladder

1. **Documented:** a clear runbook that people follow.
2. **Scripted:** a tested script a person runs.
3. **Suggested:** the system proposes the action and a person approves.
4. **Automatic:** low-risk, well-understood actions run by themselves with limits.

Only move up when the previous step has run reliably for a while.

A guarded action

```
import sys

def restart_unhealthy(service, state, dry_run=True):
    if not state.metrics_available():                    # do not act on missing data
        return "skip: metrics unavailable"
    unhealthy = state.unhealthy_instances(service)
    if len(unhealthy) > 0.2 * state.total(service):      # too many: likely a wider problem
        return "skip: more than 20% unhealthy, page a human"
    if state.restarts_last_hour(service) >= 3:           # rate limit
        return "skip: restart limit reached"
    for i in unhealthy[:1]:                              # one at a time
        print("would restart" if dry_run else "restarting", i)
        if not dry_run:
            state.restart(i)
    return "ok"
```

Guardrails

- **Preconditions** use several independent signals, never one.
- **Blast radius** limits: one instance at a time and a maximum per hour.
- **Circuit breaker:** stop automation after repeated failures.
- **Dry run** mode for testing, and **audit log** for every action.
- **Rollback** or a clear manual undo step.

Review automated actions weekly and delete the ones nobody trusts.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Runbook automation and guardrails fits together

Part	What it does	What goes wrong
The ladder	Documented: a clear runbook that people follow.	Skipping dry runs.
A guarded action	.	No limit on how many things change at once.
Guardrails	Preconditions use several independent signals, never one.	Automation with no audit log.

Flow: The ladder to A guarded action / A guarded action to Guardrails

Incident: The restart loop

Incident: *Auto-restart hit a healthy fleet when the metrics system failed.*

```
metrics: no data for 6 minutes
automation: instances unhealthy: 100%
restarted 48 of 48 instances
```

The script read missing data as failure and restarted everything, causing the outage it was meant to prevent. The team added a data-available check, a 20 percent limit, a one-at-a-time rule and a circuit breaker that pages a human.

Common mistakes

- Acting on a single signal
- No limit on how many things change at once
- Skipping dry runs
- Automation with no audit log

Check yourself

What should automation check first?

Answer: That its input data is valid

What does a rate limit do?

Answer: Caps how often an action runs

What is a circuit breaker?

Answer: It stops automation after repeated failure

What is the safest first automation step?

Answer: A tested script a person runs

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 13. Statistics for ops: percentiles, seasonality and correlation

14 minutes. The small set of statistics every AIOps tool relies on: why averages mislead, how percentiles and distributions behave, what seasonality is, and why correlation is not cause.

Every AIOps feature, from anomaly detection to root-cause hints, is statistics wearing an operations badge. If you understand five ideas, you can judge any tool's output and catch it when it is wrong.

What you will learn

- Why the average hides the users who suffer most
- How to read percentiles and what happens when you average them
- What seasonality, trend and noise are
- Why correlation between two graphs is weak evidence of cause
- How sample size and cardinality change what you can conclude

Averages hide the tail

Latency is not bell-shaped. Most requests are fast and a few are very slow, so the distribution has a long right tail. The mean is pulled up by the tail but still describes nobody: half the users are faster, and the slowest are far worse. Use percentiles. The median (p50) is the typical request. The p95 and p99 show what the unlucky users see, and a system with many calls per page makes the tail common, because a page that makes 20 calls hits the p95 of at least one of them most of the time.

Percentiles do not add up

You cannot average percentiles. If one server reports a p99 of 200 ms and another 800 ms, the fleet p99 is not 500 ms. To combine them you need the raw values or a histogram with shared buckets, then compute the percentile on the merged data. This is why Prometheus histograms and `histogram_quantile` matter, and why a dashboard that averages per-host p99 values is quietly wrong.

Seasonality, trend and noise

A metric is usually three things added together. A **trend** is the slow drift, such as steady growth. **Seasonality** is the repeating pattern: daily peaks, weekday against weekend, month-end batch jobs. **Noise** is what is left. Anomaly detection works by removing the first two and asking whether the remainder is unusual. A detector that ignores seasonality pages you every Monday morning.

Variance and robust measures

The standard deviation is itself dragged around by outliers, so one huge spike widens the band and hides the next spike. Robust measures such as the median and the median absolute deviation (MAD) are far less affected. When your data has spikes, which operations data nearly always does, prefer them.

Correlation is not cause

Two lines that rise together might share a cause, one might cause the other, or the match might be chance. With thousands of metrics, some pairs will line up by luck. Treat correlation as a way to choose where to look, not as a conclusion. Time order helps: a cause comes before its effect. Better still, a change you made (a deploy, a flag) is evidence you can check by reversing it.

Samples, cardinality and base rates

A rate computed from 10 requests is unstable. Flagging a 50 percent error rate on a service that served 4 requests is noise. Set a minimum volume before you alert. Also keep the base rate in mind: if real incidents are rare, even a detector that is 99 percent accurate produces mostly false alarms.

```
# p99 latency across the whole fleet, from shared histogram buckets
histogram_quantile(
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  0.99,
  sum by (le) (rate(http_request_duration_seconds_bucket{service="checkout"}[5m]))
)
```

This merges the buckets from every pod first and then computes the percentile, which is the correct order. Averaging per-pod p99 values would not be.

Diagram: Splitting a metric into parts a detector can reason about

Part	What it does	What goes wrong
Raw metric	Requests per second as measured.	Judging it as a single line hides structure.
Trend	The slow drift, such as growth over weeks.	Mistaken for an anomaly when it is just growth.
Seasonality	Daily and weekly repeating patterns.	Ignored, so every busy hour pages someone.
Remainder	What is left after removing trend and seasonality.	Spikes inflate the spread if you use the standard deviation.
Detector	Flags remainders that are unusually large.	A tiny sample makes the rate meaningless.
Alert or hint	A page, a ticket or a hint to an engineer.	Treated as proof of cause rather than a lead.

Flow: Raw metric to Trend / Raw metric to Seasonality / Raw metric to Remainder / Trend to Detector / Seasonality to Detector / Remainder to Detector / Detector to Alert or hint

Incident: The p99 that was an average

Incident: *A latency dashboard showed a healthy p99 of 320 ms while support received complaints about slow checkouts.*

```
$ promtool query instant http://prom:9090 'avg(p99_latency_ms{service="checkout"})'
{} => 320
$ promtool query instant http://prom:9090 'max(p99_latency_ms{service="checkout"})'
{} => 2900
(this is a simulated illustration, not a real incident)
```

The panel averaged the p99 of 12 pods. One pod had a bad disk and a p99 of 2.9 seconds, and eleven healthy pods diluted it. The team changed the panel to a merged-histogram percentile plus a per-pod maximum, and the slow pod showed at once.

Common mistakes

- Averaging percentiles across hosts
- Using the mean for latency
- Reading a correlation as a cause
- Alerting on rates from tiny samples
- Using standard deviation on spiky data

Check yourself

Why is the mean a poor summary of latency?

Answer: The distribution has a long tail that the mean blurs

How do you get a correct fleet-wide p99?

Answer: Merge histogram buckets and compute the percentile

What is seasonality?

Answer: A repeating pattern such as daily or weekly peaks

Two graphs rise together. What can you conclude?

Answer: They are worth investigating, but cause is unproven

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 14. Hands-on anomaly detection: z-score, MAD and EWMA

16 minutes. Build three small detectors in Python, see where each one fails on spiky and seasonal data, and learn how to pick one by testing it against past incidents.

You do not need a platform to detect anomalies. Three short functions cover most useful cases, and writing them once teaches you what commercial tools are doing under the hood.

What you will learn

- Write a z-score, a MAD and an EWMA detector
- See how a spike corrupts a z-score and why MAD resists it
- Add seasonality by comparing with the same hour on earlier days
- Replay past incidents against a detector before trusting it

The z-score

The z-score says how many standard deviations a point is from the mean of a window. A value above 3 is unusual if the data is roughly bell-shaped. It is quick and easy to explain, but both the mean and the standard deviation are pulled by outliers, so a big spike inside the window makes the detector deaf to the next one.

MAD, the robust version

Replace the mean with the median and the standard deviation with the median absolute deviation. A single extreme point barely moves either. The "modified z-score" scales the result so that 3.5 is a common cut-off.

EWMA, a detector with memory

An exponentially weighted moving average gives recent points more weight. It reacts to a slow drift that a fixed window would miss, and it is cheap to update one point at a time, which suits streaming. The smoothing factor decides how fast it forgets.

```
import statistics as st

def zscore(window, x):
    mu = st.fmean(window)
    sd = st.pstdev(window) or 1e-9
    return (x - mu) / sd

def modified_z(window, x):
    med = st.median(window)
    mad = st.median([abs(v - med) for v in window]) or 1e-9
    return 0.6745 * (x - med) / mad

def ewma_flags(series, alpha=0.3, k=3.0):
    mean, var, flags = series[0], 0.0, []
    for x in series[1:]:
        diff = x - mean
        flags.append(abs(diff) > k * (var ** 0.5) and var > 0)
        mean += alpha * diff
        var = (1 - alpha) * (var + alpha * diff * diff)
    return flags

cpu = [41, 43, 40, 42, 44, 41, 95, 42, 43, 41, 40, 88]
print(round(zscore(cpu[:-1], cpu[-1]), 1), round(modified_z(cpu[:-1], cpu[-1]), 1))
```

The 95 earlier in the window inflates the standard deviation, so the z-score for the final 88 comes out at about 2.7 and stays below the usual limit of 3. The modified z-score for the same point is about 31. That difference is the whole argument for robust statistics on spiky data.

MARGA by EknathaLabs marga.eknathalabs.com

Adding seasonality

For traffic-like metrics, compare a point with the values at the same time on the previous days. Build the baseline from, say, the same hour on the last 7 days, then apply MAD to that small set. This removes most daily-cycle false alarms without a model.

Test before you trust

Collect a few weeks of a metric and a list of real incidents with start times. Run each detector over the history and count three things: incidents caught, how early, and false alarms per week. A detector that catches everything but fires 40 times a week will be ignored. Tune the threshold and the minimum duration until the false alarms are tolerable, then keep the replay as a regression test.

Diagram: Choosing and testing a detector

Part	What it does	What goes wrong
Metric history	Weeks of the series plus a list of real incident times.	Too little history to see the weekly cycle.
Z-score	Mean and standard deviation of a window.	One spike widens the band and hides the next.
MAD	Median based, resists outliers.	Scale can be zero on flat data.
EWMA	Recent points weigh more, good for drift.	Slow smoothing hides a sudden jump.
Replay on history	Score each detector against known incidents.	Tuning on the same incidents you test on.
Chosen detector	The simplest one that meets recall and false alarm goals.	No owner, so it is never retuned.

Flow: Metric history to Z-score / Metric history to MAD / Metric history to EWMA / Z-score to Replay on history / MAD to Replay on history / EWMA to Replay on history / Replay on history to Chosen detector

Incident: The detector that went deaf

> **Incident:** *A z-score detector on error rate stayed quiet through a real outage that began right after a smaller spike.*

```
$ python3 replay.py --metric errors --detector zscore --incident 2026-10-02T09:10
incident start: 09:10  first alert: none
window stddev at 09:10: 18.4 (normally 2.1)
(this is a simulated illustration, not a real incident)
```

A short spike at 09:02 had inflated the standard deviation almost ninefold, so the real outage looked normal. Switching to MAD with a 5 minute minimum duration caught the outage at 09:12, and the replay was added to the test suite.

Common mistakes

- Using a z-score on spiky data
- Choosing a threshold by feel and never replaying history
- Tuning and testing on the same incidents
- Ignoring seasonality on traffic metrics
- Alerting on one point instead of a sustained change

Check yourself

Why does MAD resist outliers better than a standard deviation?

Answer: It is built on medians, which a single extreme value barely moves

What is EWMA good at?

Answer: Reacting to slow drift with cheap streaming updates

How do you know a detector is good enough?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Replay it on past incidents and count catches, lead time and false alarms

Why test on incidents you did not tune on?

Answer: Tuning on the test set overstates how well it will do

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 15. Change-point and trend detection

13 minutes. Find the moment a metric's behaviour shifted and tell a lasting level change from a spike, using CUSUM and simple trend tests that explain what happened and when.

A spike comes and goes. A change point is different: after it, the metric lives at a new level, with a new slope or a new spread. A memory leak, a cache that stopped working and a slower dependency all show up this way. Telling the two apart, and finding when the shift began, is one of the most useful things an operations model can do.

What you will learn

- The difference between a spike, a level shift and a trend
- How CUSUM accumulates small deviations into a clear signal
- How to find the start time of a shift for correlation with changes
- How to detect a slow leak before it becomes an outage

Spike, shift and trend

A spike is a short excursion that returns to normal. A level shift is a step: latency moves from 120 ms to 180 ms and stays there. A trend is a steady slope, such as memory growing 40 MB an hour. Each needs a different response. Spikes are often noise. Shifts point at a change. Trends give you time to act before a limit is reached.

CUSUM

The cumulative sum method adds up how far each point is above a target, minus a small allowance. Normal noise cancels out and the sum stays near zero. A persistent small shift keeps adding in the same direction, and the sum climbs until it crosses a threshold. It detects shifts much smaller than a per-point threshold would, and it keeps sensitivity high without firing on single spikes.

```
def cusum(series, target, slack, limit):
    pos = neg = 0.0
    start = None
    for i, x in enumerate(series):
        pos = max(0.0, pos + (x - target - slack))
        neg = max(0.0, neg + (target - x - slack))
        if pos == 0.0 and neg == 0.0:
            start = None
        elif start is None:
            start = i
        if pos > limit or neg > limit:
            return i, start
    return None, None

latency = [120] * 30 + [135] * 20
alarm_at, began_at = cusum(latency, target=120, slack=3, limit=60)
print(alarm_at, began_at)
```

The detector alarms a few points after the step and also reports where the accumulation began, which is the best estimate of when the change happened. That start time is what you compare with your deploy log.

Estimating when it began

Knowing the change started at 14:03 turns a vague complaint into a question you can answer: what changed at 14:03? Report the estimated start rather than only the alarm time, because the alarm always lags. Allow a margin of a few minutes on either side when you search the change log.

MARGA by EknathaLabs marga.eknathalabs.com

Trends and time to limit

For a leak, fit a line to the recent window and extrapolate to the limit. If memory grows 40 MB an hour and 3.2 GB remain, you have about 80 hours. A robust fit such as the Theil-Sen slope, which takes the median of pairwise slopes, resists outliers. Alert on a short time to limit, not on the level itself, so people are told when the problem matters.

Pitfalls

Seasonality looks like a repeating shift, so remove it first. Restarts reset leaks and hide the trend, so detect over a window that spans them or look at growth between restarts. Missing data can look like a step down, so treat gaps explicitly.

Diagram: From a metric to a change time

Part	What it does	What goes wrong
Metric stream	A latency or memory series.	Gaps and restarts look like shifts.
Remove seasonality	Subtract the daily and weekly pattern.	A repeating pattern is mistaken for repeated shifts.
CUSUM	Accumulates small deviations to detect a lasting shift.	Slack set too low fires on noise.
Trend fit	A robust slope and a time to limit.	A restart resets the slope and hides a leak.
Estimated start	The time the change began, not when the alarm fired.	Searching the change log only at alarm time.

Flow: Metric stream to Remove seasonality / Remove seasonality to CUSUM / Remove seasonality to Trend fit / CUSUM to Estimated start / Trend fit to Estimated start

Incident: A leak the dashboards forgave

> **Incident:** *An API service was restarted by the platform every few days and nobody asked why, until a restart failed during a busy period.*

```
$ memtrend --service api --window 72h
slope: +38 MB/hour (Theil-Sen)
limit: 4096 MB  current: 1180 MB
time to limit: about 76 hours
(this is a simulated illustration, not a real incident)
```

The memory graph looked bounded because restarts kept resetting it. A trend fit over the span between restarts showed the steady climb, and a heap profile found a cache that never evicted entries. The team fixed the cache and alerted on time to limit instead of the raw level.

Common mistakes

- Treating every spike as a change
- Reporting the alarm time instead of the estimated start
- Fitting a trend across restarts
- Leaving seasonality in the data
- Alerting on level instead of time to limit

Check yourself

What separates a change point from a spike?

Answer: After a change point the metric stays at a new level or slope

Why is CUSUM good at small shifts?

Answer: It accumulates small deviations until they add up to a signal

Why report the estimated start of a shift?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: It lets you search the change log around the real time

How should a slow memory leak be alerted?

Answer: On projected time to the limit

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. Event deduplication, grouping and suppression rules

13 minutes. Turn a stream of raw events into a few actionable incidents using fingerprints, time windows, grouping keys and suppression rules, and keep those rules safe to maintain.

Correlation starts before any machine learning. A good rules layer removes most noise cheaply, is easy to explain, and gives later models cleaner input. This lesson covers the building blocks and the ways they go wrong.

What you will learn

- Build a stable fingerprint and use it to deduplicate
- Choose grouping keys and time windows
- Write suppression rules that expire and are reviewed
- Test a rule against history before it goes live

Fingerprints

A fingerprint is a hash of the fields that define "the same problem": the alert name, the service, the environment and the failing component, but not the timestamp, the pod name or the current value. If the fingerprint is too specific, every event looks new. If it is too broad, different problems merge. Start with the labels a human would use to say "that is the same alert".

Deduplication

When an event arrives with a known fingerprint, update the existing record: bump a counter and the last-seen time instead of creating a new alert. The responder sees one item that says "seen 41 times in 12 minutes", which is far more useful than 41 items.

Grouping

Group related alerts into one incident by a shared key and a time window. Common keys are the cluster, the service, the host or the upstream dependency. The window matters: a short wait collects the burst, a long one delays the page. Prometheus Alertmanager uses `group_wait` for the first batch and `group_interval` for later changes.

Suppression and maintenance windows

Suppression hides alerts that cannot be acted on: a host under planned maintenance, a test environment, a known issue with a ticket. Every suppression needs an owner, a reason and an expiry. A silence with no expiry is how a real outage gets ignored months later.

```
route:
  group_by: [alertname, cluster, service]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers: [env = "staging"]
      receiver: ticket-only
inhibit_rules:
  - source_matchers: [alertname = "ClusterDown"]
    target_matchers: [severity =~ "warning|info"]
    equal: [cluster]
```

Alerts are grouped by name, cluster and service. Staging alerts create tickets instead of pages. While a whole cluster is down, lower-severity alerts in that cluster are inhibited.

MARGA by EknathaLabs marga.eknathalabs.com

Testing rules

Replay a week of real events through the rule set and compare the output with what responders actually handled. Count events in, incidents out, and any incident that a human had to split or merge by hand. A rule that merges two unrelated outages is worse than one that leaves a little noise.

Keeping rules healthy

Store rules in version control with review. Add a comment with the owner and the reason. Run a monthly report of silences and rules that have not matched anything, and remove them. Rules left alone become the hidden reason an alert never arrives.

Diagram: From raw events to incidents

Part	What it does	What goes wrong
Raw events	Alerts, logs and checks from many tools.	No consistent labels to match on.
Fingerprint	Hash of the fields that define the same problem.	Including the timestamp or pod name defeats it.
Deduplicate	Update a count and last-seen time on a repeat.	Dropping repeats hides how often it happens.
Group	Combine by cluster or service within a time window.	A long window delays the page.
Suppress	Hide what cannot be acted on, with an expiry.	A silence with no owner or end date.
Incident	One item with a count, a window and a likely source.	Merging unrelated faults into one.

Flow: Raw events to Fingerprint / Fingerprint to Deduplicate / Deduplicate to Group / Group to Suppress / Suppress to Incident

Incident: The silence that outlived the maintenance

Incident: *An engineer silenced disk alerts for a database host during a migration. The silence had no end time.*

```
$ amtool silence query --expired=false
ID        Matchers              Ends At  Created By  Comment
8c1f20ab  host="db-07"          never    jsmith      migration
(this is a simulated illustration, not a real incident)
```

Three weeks later the disk on db-07 filled and no alert fired. After the postmortem the team required an end time on every silence, added a weekly report of active silences and put matchers in review.

Common mistakes

- A fingerprint that includes the timestamp or pod name
- Silences without an owner or an expiry
- Grouping windows so long that pages are late
- Deploying rules without a replay test
- Rules nobody reviews

Check yourself

What belongs in a fingerprint?

Answer: Fields that define the same problem, not timestamps or pod names

What does deduplication keep?

Answer: One record with a count and last-seen time

What must every suppression have?

Answer: An owner, a reason and an expiry

MARGA by EknathaLabs marga.eknathalabs.com

How do you test a grouping rule safely?

Answer: Replay past events and compare with what responders handled

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 17. Change correlation: linking incidents to deploys

14 minutes. Most outages follow a change. Build a change feed, match it to incident start times and rank suspects, so responders see what changed before they start guessing.

If you could ask only one question at the start of an incident, ask what changed. Studies of outages keep finding that a large share follow a deploy, a configuration edit, a flag flip or an infrastructure change. A change correlator answers that question in seconds and is often the highest-value AIOps feature you can build.

What you will learn

- What belongs in a change feed and how to collect it
- How to match changes to an incident by time and by scope
- How to rank suspects and show the evidence
- How to avoid blaming the most recent deploy by default

Build the change feed

Gather every change into one stream with the same fields: time, type, system, version or diff link, author, and the service or environment it touched. Sources include CI/CD, delivery and sync tools, feature flag services, infrastructure as code runs, database migrations, certificate rotations, autoscaler and node pool changes, and cloud provider maintenance notices. Changes you do not record cannot be correlated.

Match by time

Take the estimated start of the incident, from a change-point detector rather than the page time, and look back over a window such as 30 minutes. Include a short window after it too, because clocks differ and rollouts take time to reach every pod. Rank by closeness, but do not stop there.

Match by scope

A change only explains an incident if it can reach the failing system. Use service ownership and dependency data: a deploy of the image service is a weak suspect for a login failure unless login depends on it. Score a change higher when it touches the failing service, one of its direct dependencies, or shared infrastructure the service runs on.

Rank with evidence

Combine the signals into a score and show the reasons, not just a number: "deployed 4 minutes before the error rate rose, same service, changed a timeout setting". Responders trust a hint they can check. Include a link to the diff and a one-click path to the rollback.

```
from datetime import datetime, timedelta

def rank(changes, incident_start, service, deps, window=timedelta(minutes=30)):
    scored = []
    for c in changes:
        gap = (incident_start - c["time"]).total_seconds() / 60
        if gap < -5 or gap > window.total_seconds() / 60:
            continue
        score = 3.0 / (1.0 + max(gap, 0))
        if c["service"] == service:
            score += 2.0
        elif c["service"] in deps:
            score += 1.0
        if c["type"] in ("config", "flag"):
            score += 0.5
        scored.append((round(score, 2), c["id"], c["type"], c["service"]))
    return sorted(scored, reverse=True)

t0 = datetime(2026, 11, 3, 14, 3)
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
changes = [
    {"id": "d-881", "time": t0 - timedelta(minutes=4), "type": "deploy", "service": "checkout"},
    {"id": "f-102", "time": t0 - timedelta(minutes=22), "type": "flag", "service": "search"},
]
print(rank(changes, t0, "checkout", {"payments", "inventory"}))
```

The scoring is deliberately simple and explainable. You can tune the weights against past incidents.

Beware the latest-deploy bias

The most recent change is not always the cause. A change from three days ago can start misbehaving when traffic grows. A flag flipped by a vendor, an expired certificate or a cloud maintenance event can matter more than any of your deploys. Keep a "no recent change found" result as a valid answer, and do not force a suspect.

Close the loop

When the incident ends, record which change was the cause, if any. That label is training data for tuning the weights and the best test of whether the correlator is helping.

Diagram: Linking an incident to its likely change

Part	What it does	What goes wrong
Change feed	Deploys, config, flags, infrastructure and vendor events.	Changes that never reach the feed.
Incident start	Estimated from a change-point detector.	Using the page time, which lags.
Time match	Changes shortly before the start, with a margin after it.	Clock skew between systems.
Scope match	Does the change reach the failing service or a dependency?	Stale ownership or dependency data.
Ranked suspects	A short list with reasons, diff links and rollback.	A bare score nobody can check.
Outcome label	Record the real cause for tuning.	Never recorded, so the model never improves.

Flow: Change feed to Time match / Incident start to Time match / Time match to Scope match / Scope match to Ranked suspects / Ranked suspects to Outcome label

Incident: The flag nobody deployed

> **Incident:** *Checkout errors began at 14:03. There had been no deploys that afternoon, and the on-call engineer spent twenty minutes checking infrastructure.*

```
$ changes --around 14:03 --window 30m --service checkout
14:01  flag    enable_new_tax_api   set by pricing-team   scope: checkout
13:40  config  db-pool-size         unchanged
(this is a simulated illustration, not a real incident)
```

A pricing team had switched on a feature flag two minutes earlier. The flag service was not in the change feed, so no tool had linked it. Once flags were added as a source, the correlator listed the flag first, and the next similar incident was resolved in minutes by turning it off.

Common mistakes

- Leaving feature flags and vendor events out of the feed
- Matching on the page time instead of the estimated start
- Blaming the latest change without checking scope
- Showing a score without the reasons
- Never recording the real cause

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Why add feature flags to the change feed?

Answer: A flag flip changes behaviour without a deploy

What time should you search around?

Answer: The estimated start of the problem, with a margin after it

What does scope matching add?

Answer: It checks the change can reach the failing service

When should the tool say no recent change was found?

Answer: When nothing plausible matches, instead of forcing a suspect

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 18. Topology and dependency-aware root cause

14 minutes. Use the service graph to explain why many things fail at once: walk the dependency edges, find the shared fault, and avoid blaming the loudest alert.

During an outage the service with the most alerts is rarely the cause. It is usually a victim, loud because it sits at the top of a chain. A dependency graph lets a tool reason the way an experienced engineer does: which broken thing, if it were the cause, would explain all the others?

What you will learn

- What a dependency graph contains and where it comes from
- How to find the shared upstream cause of many failures
- How blast radius works in the other direction
- How to keep topology data fresh and what to do when it is wrong

The graph

Nodes are services, databases, queues, hosts, clusters and external providers. Edges say "A calls B" or "A runs on B". Sources are service meshes, distributed traces, cloud inventory, Kubernetes ownership, and a service catalog. Traces give the most accurate call edges because they come from real traffic. A catalog gives owners and tiers. Use both.

Finding the shared cause

Mark the failing nodes. For each candidate node, count how many failing nodes depend on it, directly or through others, and how many of its own dependencies are healthy. A node that is itself failing, with all of its dependencies healthy, and which sits upstream of many failing nodes is the best candidate. If checkout, cart and search all fail and all read from one database that is also failing, the database explains the three.

Blast radius

The same graph answers a forward question: if this component fails, what is affected? Blast radius estimation helps prioritise during an incident ("this affects 14 services and 3 customer-facing flows") and supports change reviews before the incident ever happens.

Health is not binary

A dependency can be degraded: slow but not down. Use latency and error signals per edge, not just up and down. Retries and timeouts can also move the symptom: a slow database makes callers time out, retry, and overload a third service that has nothing wrong with it. Rank by where the first abnormal signal appeared in time as well as by position in the graph.

```
def ranked_causes(edges, failing):
    """edges: {service: [dependencies]}; failing: set of unhealthy services."""
    def downstream_failing(node):
        seen, stack = set(), [node]
        while stack:
            cur = stack.pop()
            for svc, deps in edges.items():
                if cur in deps and svc in failing and svc not in seen:
                    seen.add(svc)
                    stack.append(svc)
        return seen
    out = []
    for node in failing:
        healthy_deps = [d for d in edges.get(node, []) if d not in failing]
        deps = edges.get(node, [])
        if len(healthy_deps) == len(deps):
            out.append((len(downstream_failing(node)), node))
    return sorted(out, reverse=True)
```

MARGA by EknathaLabs marga.eknathalabs.com

```
edges = {
    "web": ["checkout", "search"],
    "checkout": ["orders-db", "payments"],
    "search": ["orders-db"],
    "payments": [],
    "orders-db": [],
}
print(ranked_causes(edges, {"web", "checkout", "search", "orders-db"}))
```

The function keeps only failing nodes whose own dependencies are all healthy, then ranks them by how many failing services depend on them. In this example the database explains three failures and is ranked first.

When topology is wrong

Graphs go stale: services are renamed, new calls appear, a team adds a direct database connection that bypasses the usual path. Compare the catalog with trace data regularly and report edges that exist in only one. Show the responder the path the tool used, so a wrong edge is easy to spot and report. A hint that explains itself earns trust.

Diagram: Walking the dependency graph to a shared cause

Part	What it does	What goes wrong
Web frontend	Customer facing, shows errors first.	Loudest alert, but only a victim.
Checkout	Calls orders-db and payments.	Fails because its database fails.
Search	Reads from orders-db.	Fails for the same reason.
Payments	Healthy in this incident.	Wrongly blamed because checkout calls it.
Orders database	Shared by checkout and search.	The real shared cause.

Flow: Web frontend to Checkout / Web frontend to Search / Checkout to Orders database / Search to Orders database / Checkout to Payments

Incident: Blaming the loudest service

> **Incident:** *The web frontend paged first with the highest error count, and the first fifteen minutes went to its team.*

```
$ rcagraph explain --incident INC-4412
failing: web, checkout, search, orders-db
candidates with healthy dependencies: orders-db (explains 3), payments (explains 0)
first abnormal signal: orders-db p99 latency, 09:41:10
(this is a simulated illustration, not a real incident)
```

The graph pointed to the shared database, and its latency rose first. A runaway report query was holding locks. Ending the query cleared the incident, and the team added the database to the top of the incident dashboard.

Common mistakes

- Treating the loudest service as the cause
- Using a stale catalog as the only source of edges
- Modelling health as up or down only
- Ignoring when each signal first appeared
- Hiding the path the tool used

Check yourself

Which node is the best root-cause candidate?

Answer: A failing node with healthy dependencies that many failing services depend on

What is blast radius?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: The set of components affected if one fails

Why are traces a good source of edges?

Answer: They record real calls from real traffic

Why show the path behind a hint?

Answer: So a wrong edge or a stale graph is easy to spot

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 19. Tuning alerts with data: precision, recall and burn rates

14 minutes. Measure alert quality with precision, recall, MTTA and MTTR, and use SLO burn-rate alerts as the strong non-ML baseline every smarter detector must beat.

An alert is a prediction that a person needs to act. Like any prediction it can be right or wrong in two ways, and you can measure both. Teams that measure tune faster and argue less.

What you will learn

- Define precision, recall and a useful alert-quality scorecard
- Label alerts as actionable or not from tickets and responder notes
- Write multi-window burn-rate alerts for SLOs
- Decide when a machine-learning detector is worth adding

Precision and recall for alerts

Precision is the share of alerts that were worth a response: actionable alerts divided by all alerts. **Recall** is the share of real problems that produced an alert: detected incidents divided by all incidents, including those users reported first. High precision with low recall means quiet pagers and missed outages. High recall with low precision means alert fatigue. You usually trade one against the other, and the aim is a deliberate balance rather than an accident.

Labelling

You need labels to measure. Cheap sources: did the responder take an action, did the alert link to an incident, did it auto-resolve before anyone looked, was it acknowledged and closed as "no action needed". Add a one-click "not actionable" button to the page and review those weekly. Even rough labels beat none.

A scorecard

Track per service and per team: alerts per week, share actionable, share that auto-resolved in under 5 minutes, time to acknowledge (MTTA), time to resolve (MTTR), and incidents first reported by users. Review the worst five alert rules every week and fix, merge or delete them.

Burn-rate alerts

A service level objective (SLO) gives you a principled alert. The **burn rate** is how fast you are using the error budget compared with the rate that would use it exactly over the period. A burn rate of 1 spends the budget just in time. A burn rate of 14.4 over one hour spends about 2 percent of a 30-day budget. Pair a long and a short window: the long window shows the problem is real, the short one shows it is still happening, so the alert clears quickly after recovery.

```
# Page: 2% of a 99.9% monthly budget burning in 1 hour (burn rate 14.4)
(
  sum(rate(http_requests_total{service="checkout",code=~"5.."}[1h]))
  / sum(rate(http_requests_total{service="checkout"}[1h]))
) > (14.4 * 0.001)
and
(
  sum(rate(http_requests_total{service="checkout",code=~"5.."}[5m]))
  / sum(rate(http_requests_total{service="checkout"}[5m]))
) > (14.4 * 0.001)
```

A slower ticket-level alert can use a burn rate of 3 over 6 hours with a 30 minute short window. These alerts fire on user impact, need no learned model and are easy to explain.

MARGA by EknathaLabs marga.eknathalabs.com

When to add machine learning

Treat burn-rate alerts as the baseline. Add a learned detector only where the baseline misses real incidents (recall problem) or where it cannot express the symptom, such as a leading indicator or a metric with no SLO. Measure the new detector against the baseline on the same history. If it does not catch more incidents or reduce noise, it is not worth running.

Close the loop

Every false positive and miss is a lesson. Add the missed incident to the replay set, change the rule, and rerun the replay to confirm the change helps without hurting others.

Diagram: A measurement loop for alert quality

Part	What it does	What goes wrong
Alerts fired	Every page and ticket.	No labels, so no measurement.
Label outcome	Actionable, auto-resolved or no action.	Labels only from the loudest incidents.
Scorecard	Precision, recall, MTTA, MTTR, alerts per week.	Averages hide the worst rules.
Burn-rate baseline	SLO alerts on user impact.	Using one window only.
Tune or retire	Fix the worst five rules each week.	Changes without a replay test.

Flow: Alerts fired to Label outcome / Label outcome to Scorecard / Label outcome to Burn-rate baseline / Scorecard to Tune or retire / Burn-rate baseline to Tune or retire / Tune or retire to Alerts fired

Incident: Ninety percent noise

> **Incident:** *A team's pager was quiet at night but nobody trusted it, and a real outage sat unacknowledged for eleven minutes.*

```
$ alertreport --team payments --days 30
alerts: 412  actionable: 38 (9%)  auto-resolved under 5m: 301 (73%)
incidents first reported by users: 4 of 11
worst rule: HighCPU  fired 188 times  actionable 0
(this is a simulated illustration, not a real incident)
```

One CPU rule produced almost half of all alerts and never needed action. The team deleted it, added a burn-rate alert for the checkout SLO, and within a month the actionable share rose above 60 percent while user-reported incidents dropped.

Common mistakes

- Measuring alert counts instead of actionable share
- Not counting incidents users reported first
- Using one burn-rate window
- Adding ML without comparing it with the baseline
- Tuning rules without replaying history

Check yourself

What does alert precision measure?

Answer: The share of alerts that were worth a response

What does low recall mean?

Answer: Real problems happen without an alert

Why use two windows in a burn-rate alert?

Answer: The long one shows significance and the short one shows it is ongoing

When should you add a machine-learning detector?

Answer: When it measurably beats the SLO baseline on past incidents

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 20. Multivariate and service-level anomalies

14 minutes. Look at several signals together, from error rate and latency to saturation across a fleet, using peer comparison, ratios and simple multivariate scores, while keeping results explainable.

One metric at a time misses problems that only show in combination, and it floods you when a hundred hosts each have their own graph. Moving up a level, to a service or a fleet, gives fewer and more meaningful signals.

What you will learn

- Compare an instance with its peers instead of with its own history
- Combine signals into service-level health with ratios and the golden signals
- Use an isolation forest on a small set of features
- Keep multivariate results explainable

Peers beat history for fleets

If 40 identical pods serve the same traffic, any pod that differs from the other 39 is suspicious, whatever its own history says. Peer comparison needs no training and survives deploys and traffic changes that would break a per-pod baseline. Compute the median and MAD across pods at each moment and flag the ones far from it.

Ratios and rates

A raw count rises with traffic. Ratios cancel it out: errors divided by requests, cache misses divided by lookups, retries divided by calls. A ratio that jumps while traffic is flat or falling is a stronger signal than any count.

Golden signals as one view

For each service, combine latency, traffic, errors and saturation. A service is healthy when all four are in range. The pattern matters: rising latency with flat traffic and rising saturation suggests a resource limit, while rising errors with normal latency suggests a failing dependency or bad deploy. Writing these patterns as rules gives you explainable hints without any model.

Isolation forest

An isolation forest builds random decision trees and measures how few splits are needed to isolate a point. Unusual points need fewer. It works on several features at once, needs no labels and trains quickly. Use a handful of well-chosen features per service, such as error ratio, p95 latency, CPU and queue depth, scaled to comparable ranges.

```
import statistics as st

def peer_outliers(values, k=4.0):
    """values: {instance: metric}. Flag instances far from the fleet median."""
    med = st.median(values.values())
    mad = st.median([abs(v - med) for v in values.values()]) or 1e-9
    return {n: round(0.6745 * (v - med) / mad, 1)
            for n, v in values.items()
            if abs(0.6745 * (v - med) / mad) > k}

p99 = {f"pod-{i}": 210 + (i % 5) * 6 for i in range(1, 12)}
p99["pod-12"] = 1480
print(peer_outliers(p99))
```

Only `pod-12` is flagged, with a high score, while the normal pods vary within a narrow band. The same function works for any per-instance metric.

Keeping it explainable

A score of 0.91 tells a responder nothing. Report which features contributed most and by how much compared with the peers: "pod-12: p99 latency 7 times the fleet median, error ratio normal, CPU normal". If you use a model, pair it with a plain-language summary and the underlying graphs. When people cannot see why a flag was raised, they stop trusting it.

Choosing features carefully

Too many features, or features that always move together, give noisy scores. Pick signals that describe different failure modes and check them against past incidents. Remove any feature that never mattered.

Diagram: From per-instance noise to a service-level signal

Part	What it does	What goes wrong
Instance metrics	Hundreds of per-pod and per-host series.	Alerting on each one separately.
Peer comparison	Compare each instance with the fleet median.	Peers that are not truly identical.
Ratios	Errors per request, retries per call.	Dividing by tiny traffic.
Golden signals	Latency, traffic, errors, saturation per service.	A missing signal hides a failure mode.
Isolation forest	Scores unusual feature combinations.	Features that always move together.
Explanation	Which features and how far from normal.	A bare score no one can check.

Flow: Instance metrics to Peer comparison / Instance metrics to Ratios / Peer comparison to Golden signals / Ratios to Golden signals / Golden signals to Isolation forest / Golden signals to Explanation / Isolation forest to Explanation

Incident: One bad pod in twelve

Incident: *Overall latency looked fine, yet one customer in twelve saw slow responses for two days.*

```
$ peercheck --service api --metric p99_ms
fleet median: 222  MAD: 6
outliers (modified z > 4): pod-12 (1480 ms, z=141)
(this is a simulated illustration, not a real incident)
```

The service average hid a single pod with a failing network interface. A per-pod baseline had never fired because the pod's history was always the bad one after a recent restart. Peer comparison flagged it at once, and the pod was replaced.

Common mistakes

- Alerting on each instance's own history in a fleet
- Using raw counts instead of ratios
- Feeding many correlated features to a model
- Reporting a score without the contributing features
- Treating peers as identical when they are not

Check yourself

Why compare an instance with its peers?

Answer: It needs no history and survives deploys and traffic changes

Why prefer ratios to counts?

Answer: They cancel the effect of traffic volume

What is an isolation forest good for?

Answer: Finding unusual combinations of several features without labels

What should an explainable multivariate result show?

Answer: Which features differ and by how much from normal

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: AIOps

Practical rules for anomaly detection, alert grouping, incident hints and safe auto-remediation.

Before you buy or build

- Write down a baseline: alerts per incident, time to acknowledge, time to find cause, false pages.
- Fix telemetry, service names and the change log first.
- Trial beside current practice for a few weeks. Keep it only if the numbers improve.

Anomaly detection ladder

Step	Method	Good for
1	Static threshold	Hard limits such as disk full
2	Rolling mean and spread	Metrics without a daily pattern
3	Seasonal baseline	Traffic, with daily and weekly rhythm
4	Forecast or model	Many series, after simple methods fail

```
sum(rate(http_requests_total{job="checkout"}[5m]))
/
sum(rate(http_requests_total{job="checkout"}[5m] offset 1w))
```

Reduce alert noise

```
route:
  group_by: [cluster, service]
  group_wait: 30s
inhibit_rules:
  - source_matchers: [alertname = "NodeDown"]
    target_matchers: [severity = "warning"]
    equal: [cluster, node]
```

Warning: *Silences need an expiry, a narrow matcher and a reason.*

Root cause hints

1. List changes just before the first symptom.
2. Overlay dependency health.
3. Rank by time gap, graph distance and size of anomaly.
4. Show evidence with every hint.
5. A person confirms before any production change.

Safe auto-remediation checklist

```
preconditions: [alert_still_firing, no_active_deploy, target_count_max: 2]
limits: {max_per_hour: 2, cooldown: 30m, kill_switch: automation/global}
verify: {after: 5m, expect: "queue_depth < 100"}
on_verify_fail: page_human
```

Why it matters: *Automation acts fast and at scale. Limits and a kill switch contain mistakes.*

Check an assistant's answer

1. Ask which evidence the claim rests on, and open it.
2. Turn the cause into a query you can run.
3. Line up alerts, deploys and changes on one timeline.
4. Record hits and misses.
5. A person approves every action.

MARGA by EknathaLabs
marga.eknathalabs.com

Warning: *A fluent answer is not evidence. A claim with no source is a guess.*

Warning: *Do not run a suggested rollback or restart before testing the claim against the data.*

Warning: *Do not give an AIOps tool write access before it has earned trust on read-only work.*

Warning: *Anomaly detection fires on harmless change too. Tune windows and keep the alert on user impact.*

Commands for checking a claim

```
$ promtool query instant http://prometheus:9090 'sum(rate(http_requests_total{code=~"5.."}[5m]))'

$ git log --since="2 hours ago" --oneline

$ kubectl rollout history deployment/checkout
```

MARGA by EknathaLabs
marga.eknathalabs.com